

Graph Traversal and Shortest Paths

Subject: Data Structures Topic: Graphs

Overview

An introduction to graph representations, breadth-first search, depth-first search, and shortest paths

Graph representation

- A graph is $G = (V, E)$; edges may be directed or undirected and may carry weights.
- An adjacency list uses $O(V + E)$ space and is efficient for sparse graphs.
- An adjacency matrix uses $O(V^2)$ space and gives $O(1)$ edge-existence checks.

Breadth-first and depth-first search

- BFS uses a queue, marks vertices when enqueued, and finds shortest paths by edge count in an unweighted graph.
- DFS uses recursion or an explicit stack and supports connected-component, cycle, and topological-order algorithms.
- Both traversals run in $O(V + E)$ with adjacency lists; store a visited set to avoid revisiting cycles.

Shortest paths

- Dijkstra's algorithm finds nonnegative-weight shortest paths; with a binary heap it is typically $O((V + E) \log V)$.
- Bellman-Ford supports negative edges and detects reachable negative cycles in $O(VE)$.
- For an unweighted graph, BFS gives shortest paths in $O(V + E)$; reconstruct a route using predecessor links.

Worked checks

- Initialize source distance to zero and all other distances to infinity.
- Relax edge (u, v, w) when $\text{distance}[u] + w$ is smaller than $\text{distance}[v]$.
- Do not use Dijkstra when negative edge weights are present.